



Projekt badania i badanie pilotażowe

Metody Badawcze w Informatyce

Kanstantsin Chyzheuski 211984

Julia Konarzewska 212038

Elvira Oliinyk 211994

Spis treści

1. Projekt badawczy	4
1.1 Tytuł	4
1.2 Promotor.....	4
1.3 Cele i krótki opis	4
2. Research design	5
2.1. Cel badania.....	5
2.2. Luka badawcza.....	5
2.3. Pytania badawcze	6
2.4. Hipotezy badawcze	6
2.5. Przedmioty badawcze i próba	7
2.6. Operacjonalizacja – zmienne	7
2.7. Metody badawcze.....	8
2.8. Narzędzia badawcze.....	9
2.9. Oczekiwane wyniki.....	10
2.10. Zagrożenia dla trafności.....	11
2.11. Plan badań	12
2.12. Cele publikacyjne.....	13
3. Badanie pilotażowe	14
3.1. Podmioty badawcze	14
3.2. Przebieg badania	15
3.2.1 Przygotowanie środowiska badawczego.....	15
3.2.2 Realizacja badania pilotażowego	16
3.3 Wyniki.....	16

3.3.1 Wyniki badania pilotażowego.....	16
3.3.2 Analiza jakościowa wyników	17
3.4. Wnioski z badania pilotażowego	18
3.4.1 Trafność metodyki przeprowadzonego badania	18
3.4.2 Trafność wykorzystanych narzędzi.....	18
3.4.3 Planowane modyfikacje	19
4. Wnioski	19
4.1 Wnioski projektowe	19
4.2 Wnioski ze studium pilotażowego	20
5. Literatura.....	21
Załącznik nr 1 – Scenariusz badania pilotażowego	24

1. Projekt badawczy

1.1 Tytuł

CG/CI/CD: Automatyczne generowanie kodu z artefaktów projektowych zintegrowane z potokami ciągłej integracji i ciągłego wdrażania

1.2 Promotor

dr inż. Andrzej Wardziński

1.3 Cele i krótki opis

Celem niniejszego projektu badawczego jest zbadanie, czy artefakty projektu oprogramowania (np. diagramy UML, elementy backlogu, specyfikacje systemowe lub wyniki narzędzi CASE) mogą być efektywnie przekształcane w gotowy do wdrożenia kod źródłowy za pomocą automatycznego generowania kodu, zintegrowanego bezpośrednio z potokami CI/CD.

Projekt wprowadza koncepcję **CG/CI/CD (Continuous Generation / Continuous Integration / Continuous Deployment)**, która rozszerza tradycyjne podejście DevOps poprzez dodanie wcześniejszego etapu generowania przed integracją. Etap ten automatycznie przekształca artefakty na poziomie projektu w wykonywalny kod, zmniejszając nakład pracy ręcznej, zwiększając szybkość wdrożeń oraz poprawiając spójność między projektem a implementacją.

Badanie opiera się na wynikach przeprowadzonego przeglądu systematycznego literatury (SLR), który wykazał istotną lukę badawczą: podczas gdy wiele prac koncentruje się oddzielnie na generowaniu kodu lub automatyzacji CI/CD, niewiele z nich łączy oba te aspekty w jeden w pełni zintegrowany proces wytwarzania oprogramowania.

2. Research design

2.1. Cel badania

Celem niniejszego badania jest analiza możliwości integracji automatycznego generowania kodu z pipeline'ami CI/CD w ramach koncepcji CG/CI/CD na podstawie działającego prototypu systemu.

Badanie ma na celu sprawdzenie, w jaki sposób wykorzystanie artefaktów projektowych (model YAML) jako głównego źródła może wpłynąć na automatyzację procesu wytwarzania oprogramowania, jego efektywność oraz jakość.

Dodatkowo celem jest ocena wpływu tego podejścia na proces wytwarzania oprogramowania, w szczególności na czas realizacji pipeline'u, liczbę błędów oraz konieczność ręcznej ingerencji. Badanie obejmuje również sprawdzenie, czy zmiany w modelu prowadzą do poprawnej i automatycznej regeneracji aplikacji.

2.2. Luka badawcza

Na podstawie przeprowadzonego SLR można zauważyć, że temat automatycznego generowania kodu oraz pipeline'ów CI/CD jest coraz częściej poruszany w badaniach naukowych. W szczególności widoczny jest rosnący trend wykorzystania podejść takich jak model-driven engineering oraz sztucznej inteligencji.

Jednak analiza literatury pokazuje istotną lukę badawczą:

- Większość prac koncentruje się na generowaniu kodu lub automatyzacji pipeline'ów CI/CD jako oddzielnych zagadnieniach. Niewiele badań łączy oba te elementy w jeden spójny proces;
- W literaturze rzadko spotyka się rozwiązania, w których artefakty projektowe (np. modele YAML) stanowią bezpośrednie źródło dla pełnego pipeline'u CI/CD;
- Istnieje luka w badaniach nad tym, jak automatycznie aktualizować wygenerowany kod w pipeline, gdy projektant zmieni tylko fragment diagramu;
- W wielu przypadkach brakuje także praktycznych eksperymentów, które pokazują, jak takie podejście działa w rzeczywistych warunkach.

Dodatkowo integracja generowania kodu bezpośrednio w pipeline'ach CI/CD nadal nie jest standardem i pozostaje obszarem wymagającym dalszych badań. W związku z tym istnieje potrzeba przeprowadzenia badań, które sprawdzą możliwość stworzenia zintegrowanego podejścia CG/CI/CD, łączącego generowanie kodu z pipeline'ami CI/CD oraz oceniają jego wpływ na proces wytwarzania oprogramowania.

2.3. Pytania badawcze

Na podstawie zidentyfikowanej luki badawczej sformułowano następujące pytania badawcze:

- RQ1: Czy możliwe jest automatyczne generowanie kodu z modelu projektowego (YAML) w pipeline'ie CI/CD bez ręcznej ingerencji?
- RQ2: Czy zmiana w modelu powoduje automatyczną i poprawną aktualizację kodu w pipeline'ie?
- RQ3: Jak często wygenerowany kod wymaga ręcznej poprawy przed wdrożeniem?
- RQ4: Jaki wpływ ma automatyczne generowanie kodu na liczbę błędów w procesie build i test?
- RQ5: Jakie problemy techniczne pojawiają się podczas integracji generowania kodu z pipeline'em CI/CD?

2.4. Hipotezy badawcze

Na podstawie sformułowanych pytań badawczych przyjęto następujące hipotezy:

- H1: Możliwe jest automatyczne generowanie kodu z modelu projektowego YAML w pipeline'ie CI/CD bez konieczności ręcznej ingerencji w większości przypadków.
- H2: Zmiana w modelu YAML powoduje automatyczną i poprawną aktualizację wygenerowanego kodu w pipeline'ie CI/CD.
- H3: Wygenerowany kod wymaga ręcznej poprawy w mniej niż 20% przypadków.
- H4: Wykorzystanie automatycznego generowania kodu zmniejsza liczbę błędów pojawiających się w procesie build i test.

- H5: Integracja generowania kodu z pipeline’em CI/CD wiąże się z określonymi problemami technicznymi, jednak nie uniemożliwiają one jego zastosowania w praktyce.

2.5. Przedmioty badawcze i próba

Badaniem objęty zostanie eksperymentalny pipeline CG/CI/CD, w którym artefakty projektowe (model.yaml) stanowią źródło automatycznego generowania kodu aplikacji.

Jako przedmiot badania wykorzystany zostanie prototyp systemu generującego aplikację typu CRUD (system zarządzania studentami), oparty na technologii Spring Boot.

Proces obejmuje:

- walidację modelu,
- generowanie kodu backendu,
- zapis kodu w repozytorium wygenerowanej aplikacji,
- uruchomienie pipeline’u CI/CD (build, test, Docker, deploy).

W ramach badania analizowane będą zmiany w modelu oraz ich wpływ na wygenerowany system.

W badaniu udział weźmie 3 uczestników, którzy będą wykonywać przygotowane scenariusze testowe oraz oceniać działanie systemu.

Kryteria kwalifikacyjne obejmują:

- podstawową znajomość programowania oraz pracy z systemami kontroli wersji (Git);
- umiejętność uruchamiania projektów typu Spring Boot;
- brak wcześniejszego kontaktu z badanym rozwiązaniem (w celu uniknięcia uprzedzeń).

Dobór próby ma charakter celowy i obejmuje użytkowników posiadających zdolność tworzenia modeli systemu oraz z abstrakcyjnymi artefaktami projektowymi, bez konieczności posiadania kompetencji programistycznych.

2.6. Operacjonalizacja – zmienne

W celu przeprowadzenia badania zdefiniowano następujące zmienne:

Zmienne niezależne:

- wykorzystanie automatycznego generowania kodu w pipeline'ie (tak / nie);
- stopień złożoności modelu projektowego (np. liczba encji i pól w modelu YAML);
- liczba oraz typ wprowadzonych zmian w modelu (dodanie encji, modyfikacja pól, zmiana struktury projektu).

Zmienne zależne:

- czas przejścia od modelu do gotowego buildu aplikacji w pipeline'ie CI/CD;
- liczba błędów w procesie build i test wygenerowanej aplikacji (Spring Boot);
- liczba przypadków wymagających ręcznej poprawy kodu;
- poprawność automatycznej regeneracji aplikacji po zmianach w modelu.

Zmienne zakłócające:

- jakość i szczegółowość modeli YAML;
- konfiguracja narzędzi (generator, CI/CD, Maven, Docker);
- jakość generatora kodu;
- środowisko uruchomieniowe.

Zmienne ukryte:

- doświadczenie osoby konfigurującej pipeline;
- ograniczenia technologii wykorzystanych w prototypie;
- niejawne zależności między elementami modelu.

2.7. Metody badawcze

W ramach badania zastosowane zostaną następujące metody badawcze:

- eksperyment w kontrolowanych warunkach – polegający na uruchamianiu pipeline'u CG/CI/CD po wprowadzeniu zmian w modelu projektowym (YAML) oraz analizie wyników (czas wykonania, błędy, ręczne

interwencji). Metoda ta pozwala odpowiedzieć na pytania RQ1, RQ2, RQ3 oraz RQ4;

- studium przypadku (case study) – analiza prototypowego systemu generującego aplikację backendową (Spring Boot) na podstawie artefaktów projektowych. Metoda ta umożliwia ocenę działania podejścia CG/CI/CD w praktyce i odnosi się do wszystkich pytań badawczych;
- ankietowanie uczestników – przeprowadzone po wykonaniu scenariuszy testowych w celu oceny użyteczności rozwiązania oraz identyfikacji problemów technicznych. Metoda ta pozwala odpowiedzieć głównie na pytanie RQ5.

Zastosowanie powyższych metod umożliwia zarówno ilościową (czas, błędy, interwencje), jak i jakościową ocenę działania podejścia CG/CI/CD.

2.8. Narzędzia badawcze

W badaniu wykorzystane zostaną następujące narzędzia badawcze:

- artefakty projektowe – model w formacie YAML, stanowiące źródło generowania kodu;
- generator kodu – implementacja w języku Python, odpowiedzialna za walidację modelu oraz generowanie aplikacji backendowej;
- repozytorium źródłowe (cg-source-repo) – zawierające artefakty projektowe oraz generator;
- repozytorium wygenerowanej aplikacji (generated-app-repo) – zawierające wygenerowany kod aplikacji;
- framework Spring Boot – wykorzystywany do budowy backendu aplikacji;
- Maven – narzędzie do budowania projektu;
- pipeline CI/CD (np. Jenkins) – realizujący proces build, test oraz integracji;
- Docker – wykorzystywany do budowy i uruchamiania aplikacji w kontenerze;
- GitHub – do zarządzania repozytoriami i integracji z pipeline’em.

Projekt eksperymentu zakłada wykonanie przez uczestników zestawu scenariuszy testowych polegających na:

- wprowadzaniu zmian w modelu YAML;
- uruchomieniu generatora oraz pipeline'u CG/CI/CD;
- obserwacji wygenerowanej aplikacji oraz wyników procesu;
- rejestrowaniu czasu wykonania, liczby błędów oraz koniecznych interwencji manualnych.

Dodatkowo w badaniu wykorzystany zostanie kwestionariusz ankiety przeznaczony dla uczestników po wykonaniu zadań:

PSQ1: Czy format YAML pliku wejściowego jest zrozumiały?

PSQ2: Czy uważasz, że takie podejście może być użyteczne w realnych projektach?

PSQ3: Czy uważasz, że stworzenie modelu w pliku YAML i automatyczna generacja kodu jest łatwiejsza niż ręczne pisanie kodu?

PSQ4: Jakie trudności lub niejasności napotkałeś podczas wykonywania zadań?

Projekt eksperymentu zakłada obserwację pełnego przepływu CG/CI/CD: od zmiany modelu, przez generowanie kodu, aż do wdrożenia aplikacji.

2.9. Oczekiwane wyniki

Oczekuje się uzyskania zarówno wyników ilościowych, jak i jakościowych.

Wyniki ilościowe:

- czas pełnego przebiegu pipeline'u (od zmiany modelu do gotowego buildu);
- liczba błędów w procesie build i test wygenerowanej aplikacji;
- liczba ręcznych interwencji w pipeline'ie;
- procent przypadków wymagających ręcznej poprawy wygenerowanego kodu.

Wyniki jakościowe:

- identyfikacja głównych problemów technicznych związanych z integracją generowania kodu z pipeline'em CI/CD;
- ocena poprawności działania mechanizmu automatycznej regeneracji kodu po zmianach w modelu;
- ocena stabilności działania pipeline'u przy zmianach w modelach UML;

- ocena użyteczności rozwiązania na podstawie opinii uczestników.

2.10. Zagrożenia dla trafności

W trakcie badania mogą wystąpić różne zagrożenia dla trafności wyników. Wyróżniono następujące ich rodzaje:

Zagrożenia dla trafności konstrukcyjnej:

- możliwość nieprecyzyjnego pomiaru zmiennych, takich jak liczba ręcznych interwencji lub liczba błędów;
- uproszczone model projektowy (YAML) może nie odzwierciedlać rzeczywistych projektów.

Sposoby minimalizacji: stosowanie jednoznacznych metryk oraz przygotowanie modeli o różnym stopniu złożoności.

Zagrożenia dla trafności wewnętrznej:

- wpływ czynników niezależnych od badanego rozwiązania, takich jak konfiguracja środowiska, generatora lub narzędzi CI/CD;
- możliwość błędów wynikających z niepoprawnej konfiguracji pipeline'u lub generatora kodu.

Sposoby minimalizacji: wykorzystanie tego samego środowiska dla wszystkich eksperymentów oraz powtarzanie testów dla różnych przypadków.

Zagrożenia dla trafności zewnętrznej:

- badanie przeprowadzane na jednym projekcie może nie być reprezentatywne dla innych systemów;
- ograniczona liczba scenariuszy testowych i modeli projektowych.

Sposoby minimalizacji: zastosowanie modeli o różnym stopniu złożoności oraz analiza kilku iteracji zmian.

Zagrożenia dla trafności wnioskowej:

- niewielka liczba przeprowadzonych eksperymentów może wpływać na wiarygodność wyników;
- możliwość błędnej interpretacji uzyskanych rezultatów.

Sposoby minimalizacji: dokładna analiza wyników, porównanie danych z różnych iteracji oraz uwzględnienie ograniczeń prototypu w interpretacji wyników.

2.11. Plan badań

Proces badania zostanie przeprowadzony etapami, aby uporządkować pracę oraz umożliwić systematyczną analizę wyników.

Na początku przygotowany zostanie prototyp systemu CG/CI/CD, obejmujący repozytorium artefaktów projektowych (model.yaml), generator kodu oraz pipeline CI/CD. Następnie zostanie skonfigurowany proces automatycznej walidacji modelu, generowania kodu aplikacji (Spring Boot) oraz jego dalszego przetwarzania (build, test, Docker).

Trzech uczestników wykona przygotowany scenariusz badawczy, obejmujący:

- pobranie repozytorium i analizę struktury projektu;
- zapoznanie się z modelem YAML oraz konfiguracją pipeline'u (Jenkinsfile);
- modyfikację modelu poprzez dodanie nowej encji;
- wysłanie zmian do repozytorium, co uruchamia pipeline;
- obserwację procesu automatycznej walidacji, generowania kodu oraz builda;
- weryfikację wygenerowanej aplikacji;
- ocenę działania systemu poprzez wypełnienie ankiety.

W kolejnym etapie zebrane dane (czas wykonania, liczba błędów, interwencje manualne oraz odpowiedzi ankietowe) zostaną przeanalizowane.

Na końcu zostaną sformułowane wnioski oraz ocena skuteczności podejścia CG/CI/CD.

Etapy procesu:

1. Przygotowanie i analiza prototypu CG/CI/CD;
2. Konfiguracja repozytoriów (cg-source-repo oraz generated-app-repo);
3. Przygotowanie scenariusza badania pilotażowego;

4. Integracja generatora kodu z pipeline'em CI/CD;
5. Wykonanie scenariuszy testowych przez trzech uczestników;
6. Zbieranie danych (czas, błędy, interwencje);
7. Analiza wyników i opracowanie wniosków.

Praca zostanie podzielona pomiędzy członków zespołu. Poszczególne osoby będą odpowiedzialne za:

- implementację generatora kodu;
- konfigurację pipeline'u CI/CD;
- przygotowanie scenariusza badania oraz ankiety;
- przeprowadzenie badania pilotażowego i analizę wyników.

W trakcie realizacji badania wykorzystane zostaną:

- repozytorium Git do zarządzania kodem;
- narzędzie CI/CD (Jenkins);
- generator kodu w Pythonie;
- framework Spring Boot oraz Maven;
- Docker do budowy i uruchamiania aplikacji;
- kwestionariusz ankiety do oceny użyteczności rozwiązania.

2.12. Cele publikacyjne

Wyniki przeprowadzonego badania mogą zostać wykorzystane do przygotowania publikacji naukowej dotyczącej integracji automatycznego generowania kodu z pipeline'ami CI/CD.

Potencjalne obszary publikacji obejmują:

- a. analizę wpływu wykorzystania artefaktów projektowych (YAML) jako źródła na czas i automatyzację procesu wytwarzania oprogramowania;
- b. ocenę skuteczności mechanizmu automatycznej regeneracji aplikacji po zmianach w modelu;
- c. identyfikację problemów technicznych zaobserwowanych podczas implementacji i testowania prototypu;
- d. ocenę możliwości zastosowania tego podejścia w praktyce.

Wyniki mogą zostać opublikowane w czasopismach lub na konferencjach związanych z inżynierią oprogramowania, DevOps oraz automatyzacją procesów wytwarzania oprogramowania, takich jak International Conference on Software Engineering, International Conference on Automated Software Engineering czy IEEE International Conference on Software Architecture.

3. Badanie pilotażowe

3.1. Podmioty badawcze

W badaniu pilotażowym wykorzystano jeden testowy projekt aplikacji backendowej Spring Boot dla domeny uniwersyteckiej, obsługującej encje Student oraz Course.

Głównym przedmiotem badania był prototyp pipeline'u CG/CI/CD składający się z dwóch repozytoriów: cg-source-repo oraz generated-app-repo. Repozytorium cg-source-repo zawierało artefakty projektowe, generator kodu oraz konfigurację pipeline'u Jenkins. Repozytorium generated-app-repo zawierało kod aplikacji wygenerowany automatycznie na podstawie modelu projektowego.

W obecnej wersji prototypu głównym artefaktem wejściowym generatora był plik model.yaml, opisujący encje oraz ich pola. Diagram PlantUML pełnił rolę wizualnego artefaktu dokumentacyjnego reprezentującego strukturę domenową systemu.

W badaniu pilotażowym wzięło udział trzech uczestników posiadających podstawową lub średnią wiedzę z zakresu programowania, systemów kontroli wersji Git, REST API oraz procesów CI/CD.

Kryteria kwalifikacji projektu testowego:

- spójne artefakty wejściowe: model YAML oraz odpowiadający mu diagram klas PlantUML,
- możliwość łatwego rozszerzenia aplikacji poprzez dodanie nowej encji do modelu,
- struktura encji w modelu pozwalająca na stworzenie dla niej operacji CRUD (Create, Read, Update, Delete).

Kryteria kwalifikacji uczestników:

- znajomość podstaw programowania aplikacji webowych,
- podstawowa znajomość systemu Git,
- podstawowa znajomość REST API,
- podstawowa znajomość działania procesu CI/CD,
- podstawowa umiejętność pracy z plikami konfiguracyjnymi, takimi jak YAML.

Kryteria wykluczenia:

- brak doświadczenia z podstawami programowania,
- brak umiejętności pracy z repozytorium Git,
- brak możliwości wykonania prostych operacji na plikach projektu.

3.2. Przebieg badania

3.2.1 Przygotowanie środowiska badawczego

Przed rozpoczęciem badania pilotażowego przygotowano eksperymentalne środowisko CG/CI/CD.

W ramach przygotowania:

- utworzono repozytorium źródłowe cg-source-repo, zawierające artefakty projektowe (model.yaml, diagramy PlantUML), generator kodu oraz konfigurację pipeline'u,
- utworzono repozytorium generated-app-repo, przeznaczone do przechowywania wygenerowanego kodu aplikacji,
- skonfigurowano lokalne środowisko Jenkins odpowiedzialne za automatyczne uruchamianie pipeline'u CG/CI/CD,
- przygotowano generator kodu w języku Python wykorzystujący bibliotekę Jinja2,
- skonfigurowano proces build aplikacji Spring Boot z wykorzystaniem Maven,
- przygotowano środowisko testowe umożliwiające uruchamianie wygenerowanych aplikacji backendowych.

Pipeline Jenkins został skonfigurowany w taki sposób, aby po wykryciu zmian w repozytorium źródłowym automatycznie:

- uruchamiał proces walidacji modelu,
- uruchamiał generowanie kodu,
- aktualizował repozytorium generated-app-repo,
- wykonywał build aplikacji backendowej.

3.2.2 Realizacja badania pilotażowego

Badanie pilotażowe zostało przeprowadzone przez autorów projektu w dniach 1–6 maja.

W badaniu uczestniczyły trzy osoby, które spełniły kryteria kwalifikacji. Badanie zostało przeprowadzone według wcześniej przygotowanego scenariusza (Załącznik 1). Badanie zostało przeprowadzone asynchronicznie - każdy z uczestników wykonywał zadania zgodnie ze scenariuszem na własnym komputerze.

Realizacja badania polegała na wykonywaniu przez uczestników kolejnych kroków opisanych w scenariuszu – najpierw każdy uczestnik pobierał repozytorium zawierające prototyp aplikacji, następnie po zapoznaniu się ze strukturą artefaktów projektowych rozpoczynał się etap modyfikacji. Po dodaniu do artefaktów projektowych nowej encji i dodaniu zmian do repozytorium, automatycznie uruchamiał się potok CG/CI/CD. Na koniec, każdy z uczestników weryfikował, czy kod został wygenerowany, a następnie uruchamiał lokalnie aplikację i testował poprawność wygenerowanego endpointu.

Po przeprowadzeniu badania, każdy z uczestników odpowiedział również na pytania ankietowe sformułowane powyżej.

3.3 Wyniki

3.3.1 Wyniki badania pilotażowego

W Tabeli 1 przedstawiono wyniki przeprowadzonego badania pilotażowego. Każdy z uczestników badania wykonał wszystkie kroki badania. Problemy w trakcie realizacji badania wystąpiły tylko u jednego uczestnika, gdzie okazało się, że nie zapisał on zmienionego pliku YAML, przez co potok CG/CI/CD nie wygenerował kodu modelu.

Tabela 1: Zestawienie wyników badania pilotażowego

ID Uczestnika	Procent wykonanych kroków	Problemy	Czas trwania pipeline'u CG/CI/CD	Czy wygenerowany kod był poprawny?
P1	100%	Brak	43 sekundy	Tak

P2	100%	Brak	51 sekund	Tak
P3	100%	Niezapisany plik YAML	47 sekund	Tak

W tabeli 2 przedstawiono opracowane odpowiedzi uczestników na pytania zadane po badaniu na temat testowanego prototypu.

Tabela 2: Zestawienie odpowiedzi na pytania zadane po badaniu

ID Uczestnika	PSQ1	PSQ2	PSQ3	PSQ4
P1	TAK	TAK	TAK	Żadnych
P2	TAK	TAK	TAK	Problem z weryfikacją poprawności kodu ze względu na niskie doświadczenie z Java SpringBoot
P3	TAK	TAK	TAK	Problemy związane z niskim doświadczeniem w korzystaniu z edytora VS Code

Wygenerowane aplikacje były poprawnie kompilowane za pomocą Maven i uruchamiane jako aplikacje Spring Boot z aktywnym REST API oraz integracją z bazą danych H2.

3.3.2 Analiza jakościowa wyników

Analiza jakościowa wyników wykazała potencjalnie wysoką użyteczność oraz skuteczność prototypowanego narzędzia CG/CI/CD. Odpowiedzi uczestników na pytanie zadane po wykonanym badaniu pokazują, że modyfikacja artefaktów projektowych jest łatwiejsza od ręcznego pisania kodu klas. Ponadto, uczestnicy

jednoznacznie zgodzili się, że format YAML pliku wejściowego jest zrozumiały oraz że takie podejście może być w przyszłości używane w realnych projektach.

Przeprowadzone badanie pilotażowe ujawniło również pewne trudności operacyjne. Uczestnicy P2 i P3 zaraportowali problemy związane z nieznaną używanymi frameworkami i narzędziami, co sugeruje, że mimo automatyzacji procesu tworzenia kodu, osoba korzystająca z oprogramowania nadal potrzebuje wiedzy domenowej na temat wykorzystywanych narzędzi.

Powyższe wyniki sugerują, że istnieje potrzeba implementacji mechanizmu automatycznego wykrywania zmian lub systemu ostrzeżeń w samym generatorze oraz że należy wzbogacić prototypowane narzędzie o silniejszy mechanizm automatycznej walidacji, aby jak najbardziej zredukować konieczność ręcznej weryfikacji kodu przez użytkownika.

Ponadto, porównanie oczekiwanych wyników z wynikami otrzymanymi pokazuje, że obecny prototyp jest zbyt prosty, aby móc prawidłowo sprawdzić poprawność jego działania. Z tego powodu należy poprawić model tak, aby był on w stanie przyjąć bardziej skomplikowane artefakty i tworzyć zaawansowane aplikacje.

3.4. Wnioski z badania pilotażowego

Badanie pilotażowe potwierdziło, że prototyp skutecznie realizuje główną koncepcję podejścia CG/CI/CD (Continuous Generation, Integration and Deployment), w której formalne artefakty projektowe pełnią rolę głównego źródła automatycznej generacji oprogramowania.

3.4.1 Trafność metodyki przeprowadzonego badania

Pomyślna realizacja badania przez użytkowników wykazała, że scenariusz eksperymentu jest zrozumiały. Jedyną pomyłką uczestnika P3 wskazuje, że należy w scenariuszu podkreślić konieczność zapisania pliku przed przesłaniem go do repozytorium zdalnego, aby zminimalizować szanse ponownego wystąpienia błędu.

3.4.2 Trafność wykorzystanych narzędzi

Za pomocą narzędzi wykorzystanych w prototypie udało się osiągnąć zamierzony cel (wygenerowany kod aplikacji) co potwierdza, że powinny być one wystarczające do stworzenia docelowego oprogramowania.

3.4.3 Planowane modyfikacje

Jak wykazała analiza jakościowa wyników, manualna weryfikacja wygenerowanego kodu podwyższa próg umiejętności wymaganych do uczestnictwa w przeprowadzonym badaniu. Z tego powodu postanowiono rozbudować mechanizm walidacji wygenerowanego kodu tak, aby weryfikacja jego poprawności w jak największym stopniu zawierała się w potoku CG/CI/CD. Dzięki temu w badaniu głównym narzędzie będzie mogło być testowane przez użytkowników z niskim poziomem wiedzy programistycznej i domenowej. Oprócz tego, dzięki analizie jakościowej zauważono, że w celu otrzymania wszystkich oczekiwanych wyników, należy rozbudować model tak, aby umożliwiał on generację bardziej zaawansowanych klas.

4. Wnioski

4.1 Wnioski projektowe

Przeprowadzone badanie pozwoliło ocenić poprawność zaprojektowanego procesu badawczego oraz jego przydatność do analizy koncepcji CG/CI/CD.

Opracowane badanie umożliwia połączenie elementów:

- Model-Driven Engineering,
- automatycznego generowania kodu,
- pipeline'ów CI/CD,
- praktycznej walidacji eksperymentalnej.

Zastosowanie eksperymentalnego prototypu pozwoliło przeprowadzić obserwację pełnego procesu transformacji artefaktów projektowych w działającą aplikację backendową.

Jednym z najważniejszych wniosków projektowych było potwierdzenie, że wykorzystanie formalnych artefaktów projektowych jako głównego źródła generacji kodu może zostać skutecznie zintegrowane z nowoczesnymi pipeline'ami automatyzacji.

Badanie wykazało również, że:

- YAML jest wystarczającym formatem do budowy pierwszej wersji prototypu,
- oddzielenie artefaktów projektowych od wygenerowanego kodu upraszcza organizację procesu,

- integracja generatora z Jenkins pipeline umożliwia automatyzację wcześniejszych etapów software delivery lifecycle.

W trakcie projektowania badania wyciągnięto również następujące wnioski:

- Zaprojektowanie badania w formie kontrolowanego eksperymentu okazało się optymalnym sposobem na weryfikację wykonalności projektu.
- Duży wpływ na przebieg badania mają zmienne ukryte, między innymi doświadczenie osoby uruchamiającej pipeline. Różny stopień doświadczenia z wykorzystanymi narzędziami i z programowaniem może fałszować pomiary zmiennych zależnych.
- Projekt badania jest bardzo ważnym elementem procesu badawczego. Staranne zaplanowanie projektu jest kluczowe dla zapewnienia wiarygodności badania.

Podsumowując, projektowanie badania jest bardzo przydatnym narzędziem w procesie badawczym. Odpowiednie wykonanie projektu badania pozwala zapewnić, że badanie przyniesie wyniki o wysokiej wartości naukowej.

4.2 Wnioski ze studium pilotażowego

Badanie pilotażowe potwierdziło techniczną wykonalność opracowanego prototypu CG/CI/CD oraz poprawność przyjętych metod badawczych.

Wszyscy uczestnicy byli w stanie:

- zmodyfikować artefakty projektowe,
- uruchomić proces automatyzacji,
- wygenerować aplikację backendową,
- zweryfikować działanie wygenerowanego REST API.

Uzyskane wyniki wskazują, że opracowany pipeline jest zrozumiały dla użytkowników posiadających podstawową wiedzę z zakresu programowania oraz procesów CI/CD.

Badanie pilotażowe wykazało również, że:

- automatyczna regeneracja aplikacji po zmianach modelu działa poprawnie,
- Jenkins może skutecznie pełnić rolę warstwy automatyzacji CG/CI/CD,
- Podejście model-driven generation może ograniczyć ilość ręcznie pisanego kodu w prostych aplikacjach backendowych.

Podczas badania zidentyfikowano również problemy praktyczne, takie jak:

- błędy wynikające z niepoprawnego przygotowania artefaktów,
- konieczność lepszej walidacji danych wejściowych przed uruchomieniem generatora.

Jedną z istotnych wyniesionych lekcji było stwierdzenie, że nawet niewielkie problemy organizacyjne (np. brak zapisania pliku YAML przed wykonaniem akcji push) mogą wpływać na poprawność działania całego pipeline'u.

Badanie pilotażowe uzasadnia kontynuację badań nad:

- rozwojem semantycznych artefaktów projektowych,
- integracją z CASE tools,
- generacją bardziej złożonych architektur aplikacyjnych,
- rozszerzeniem pipeline'u o pełną automatyzację deploymentu.

5. Literatura

Bian, Yan, Qing Zou, Weirui Yue, Kun Huang, Le Liu, i Xiao Han. „A Research of Page Automatic Publishing Scheme Based on Low Code Platform”. *2023 3rd International Conference on Electronic Information Engineering and Computer Science (EIECS)*, września 2023, 568–73.

<https://doi.org/10.1109/EIECS59936.2023.10435475>.

Da Gĩão, Hugo. „A Model-Driven Approach for CI/CD”. *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, października 2025, 52–58.

<https://doi.org/10.1109/MODELS-C68889.2025.00014>.

Fend, Andreas, i Dominik Bork. „CPSA_{ML}: A Language and Code Generation Framework for Digital Twin Based Monitoring of Mobile Cyber-Physical Systems”. *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 23 października 2022, 649–58.

<https://doi.org/10.1145/3550356.3563134>.

Ferry, Nicolas, Phu H. Nguyen, Hui Song, et al. „Continuous Deployment of Trustworthy Smart IoT Systems.” *The Journal of Object Technology* 19, nr 2 (2020): 16:1. <https://doi.org/10.5381/jot.2020.19.2.a16>.

Gĩão, Hugo da, Vasco Amaral, Gregor Engels, et al. „A Metamodel for Reengineering CI/CD Pipelines”. *2025 ACM/IEEE 28th International Conference on Model Driven*

Engineering Languages and Systems (MODELS), października 2025, 221–31.
<https://doi.org/10.1109/MODELS67397.2025.00026>.

Gunawan, Go Frendi, i Mukhlis Amien. „ZRB: A Novel Framework for Code Generation and Task Automation”. *2024 10th International Conference on Smart Computing and Communication (ICSCC)*, lipca 2024, 123–27.
<https://doi.org/10.1109/ICSCC62041.2024.10690452>.

Hugues, Jerome, Anton Hristosov, John J. Hudak, i Joe Yankel. „TwinOps - DevOps Meets Model-Based Engineering and Digital Twins for the Engineering of CPS”. *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 16 października 2020, 1–5. <https://doi.org/10.1145/3417990.3421446>.

K M, Kiran Raj, Karthik K. Poojary, Abhay, Aishwarya R S, i Lathesh Kumar S R. „AI-Driven DevOps for Intelligent Automation in Continuous Software Delivery Pipelines”. *2025 5th International Conference on Evolutionary Computing and Mobile Sustainable Networks (ICECMSN)*, listopada 2025, 433–40.
<https://doi.org/10.1109/ICECMSN68058.2025.11382867>.

Karlovs-Karlovs, Uldis. „Development of a Model-Driven DevOps Solution Based on Context-Engineered LLM Code Generation: PROFES Doctoral Symposium”. W *Product-Focused Software Process Improvement. Industry, Doctoral-Symposium, Tutorial, and Workshop Papers*, zredagowane przez Giuseppe Scanniello, Valentina Lenarduzzi, Simone Romano, Sira Vegas, i Rita Francese. Springer Nature Switzerland, 2026. https://doi.org/10.1007/978-3-032-12092-2_10.

Karlovs-Karlovs, Uldis, Oksana Nikiforova, Oscar Pastor, i Kristijans Vēveris. „MDDOAI: A Model-Driven DevOps Approach to CI/CD Automation”. *2025 66th International Scientific Conference on Information Technology and Management Science of Riga Technical University (ITMS)*, października 2025, 1–9.
<https://doi.org/10.1109/ITMS67030.2025.11236630>.

Miñón, Raúl, Josu Diaz-de-Arcaya, Ana I. Torre-Bastida, et al. „ArtifactOps and ArtifactDL: A Methodology and a Language for Conceptualizing and Operationalising Different Types of Pipelines”. *Journal of Cloud Computing* 14, nr 1 (2025): 42. <https://doi.org/10.1186/s13677-025-00761-w>.

Nagvekar, Ravi. „Agentic AI-Driven CI/CD Pipelines for Autonomous Software Delivery”. *2025 IEEE 5th International Conference on ICT in Business Industry & Government (ICTBIG)*, grudnia 2025, 1–4.
<https://doi.org/10.1109/ICTBIG68706.2025.11323919>.

Nguyen, Van-Viet, Huu-Khanh Nguyen, Kim-Son Nguyen, Minh-Hue Luong Thi, The-Vinh Nguyen, i Duc-Quang Vu. „Automated UML Generation: A Framework for

Class Diagram Synthesis and Multimodal Validation”. W *Future Data and Security Engineering*, zredagowane przez Tran Khanh Dang, Josef Küng, i Tai M. Chung. Springer Nature, 2026. https://doi.org/10.1007/978-981-95-4724-1_15.

Paniagua, Cristina, i Fernando Labra Caso. „Models2Code: Autonomous Model-based Generation to Expedite the Engineering Process”. *Systems Engineering* 28, nr 2 (2025): 224–37. <https://doi.org/10.1002/sys.21789>.

Sarschar, Mahja, Gefei Zhang, i Annika Nowak. „PACGBI: A Pipeline for Automated Code Generation from Backlog Items”. *2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, października 2024, 2338–41. <https://ieeexplore.ieee.org/document/10764968>.

Sinha, Saurabh, Tara Astigarraga, Richard B. Hull, et al. „Auto-Generation of Domain-Specific Systems: Cloud-Hosted DevOps for Business Users”. *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*, października 2020, 219–28. <https://doi.org/10.1109/CLOUD49709.2020.00041>.

Somda, Flavien Hervé, Arnold Stéphane Kabore, i Boureima Zerbo. „A Metamodel for Simplifying Infrastructure Deployment Using Vagrant”. *2025 IEEE Multi-Conference on Natural and Engineering Sciences for Sahel’s Sustainable Development (MNE3SD)*, 27 listopada 2025, 1–7. <https://doi.org/10.1109/MNE3SD67637.2025.11323237>.

Süß, Jörn Guy, Samantha Swift, i Eban Escott. „Using DevOps Toolchains in Agile Model-Driven Engineering”. *Software and Systems Modeling* 21, nr 4 (2022): 1495–510. <https://doi.org/10.1007/s10270-022-01003-2>.

Ta, Duong. „ADA-Gen: Iterative and Incremental Generation of Full-Stack Apps for Learning Agile/DevOps Software Development Practices”. *Proceedings of the 17th International Conference on Computer Supported Education*, 2025, 363–70. <https://doi.org/10.5220/0013422800003932>.

Taibi, Davide, Yuanfang Cai, Ingo Weber, et al. „Continuous Alignment Between Software Architecture Design and Development in CI/CD Pipelines”. W *Software Architecture: Research Roadmaps from the Community*, zredagowane przez Patrizio Pelliccione, Rick Kazman, Ingo Weber, i Anna Liu. Springer Nature Switzerland, 2023. https://doi.org/10.1007/978-3-031-36847-9_4.

Teumert, Sebastian, i Tim Tegeler. „Towards X-as-Models in the Context of CI/CD”. *Electronic Communications of the EASST* 84, nr Electronic Communications of the EASST, Vol. 84 (2025): 12th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation / 2nd AISoLA-Doctoral Symposium, 2024 (2025). <https://doi.org/10.14279/ECEASST.V84.2680>.

Załącznik nr 1 – Scenariusz badania pilotażowego

Celem niniejszego badania pilotażowego jest ocena wykonalności oraz zrozumiałości pełnego pipeline'u **CG/CI/CD**. Badanie weryfikuje, czy uczestnicy są w stanie wpłynąć na produkt końcowy (oprogramowanie) wyłącznie poprzez modyfikację wysokopoziomowych artefaktów projektowych, przy pełnej automatyzacji procesu generowania i wdrażania przez serwer Jenkins.

Środowisko badawcze

Uczestnicy otrzymują dostęp do:

- **Repozytorium źródłowego (cg-source-repo):** Zawiera model YAML i logikę generatora.
- **Repozytorium docelowego (generated-app-repo):** Miejsce zapisu wygenerowanego kodu.
- **Instancji Jenkins:** Serwer automatyzacji obsługujący proces CG/CI.

Procedura Badawcza (Krok po Kroku)

Krok 1: Inicjalizacja pracy

Uczestnik pobiera repozytorium źródłowe na lokalną stację roboczą:

```
Bash  
git clone [adres repozytorium]  
cd cg-source-repo
```

Oczekiwany rezultat: Pomyślne pobranie struktury projektu na dysk.

Krok 2: Analiza struktury i modelu

Uczestnik analizuje separację komponentów w folderze artifacts/ (model YAML i dokumentacja PlantUML) oraz zapoznaje się z plikiem Jenkinsfile. Następnie otwiera plik artifacts/model.yaml w celu zrozumienia definicji encji.

Krok 3: Modyfikacja artefaktu (Zadanie główne)

Uczestnik dodaje nową encję Teacher do modelu. **Uwaga:** Należy pamiętać o zapisaniu pliku w edytorze (np. VS Code) przed przejściem dalej.

YAML

Teacher:
fields:
 id: Long
 name: String
 department: String

Krok 4: Uruchomienie automatyzacji (Push & Automate)

Uczestnik wysyła zmiany do zdalnego repozytorium:

```
Bash  
git add .  
git commit -m "Add Teacher entity"  
git push
```

Oczekiwany rezultat: Jenkins automatycznie wykrywa zmiany, uruchamia walidację, generuje kod Java i wykonuje build aplikacji Spring Boot.

Krok 5: Weryfikacja efektów i test API

Uczestnik sprawdza repozytorium generated-app-repo, czy pojawiły się nowe klasy (np. TeacherController.java), a następnie uruchamia aplikację i testuje endpoint:

<http://localhost:8080/api/teachers>

Oczekiwany rezultat: Otrzymanie odpowiedzi [], co potwierdza poprawne działanie wygenerowanego REST API.